

Ensemble models (2-qism)

- Ensemble learning ^{Bir xil ma'no} == Ensemble Algorithms?
- Linear learning ^{Bir xil ma'no} == Linear Algorithms?

• Nima uchun Ensemble learning (Algorithms)

① Single algorithms (DT, LR...) overfitting-ga yoki yaroqli natija chiqarish u-n kuchsizlik qilishi mumkin.

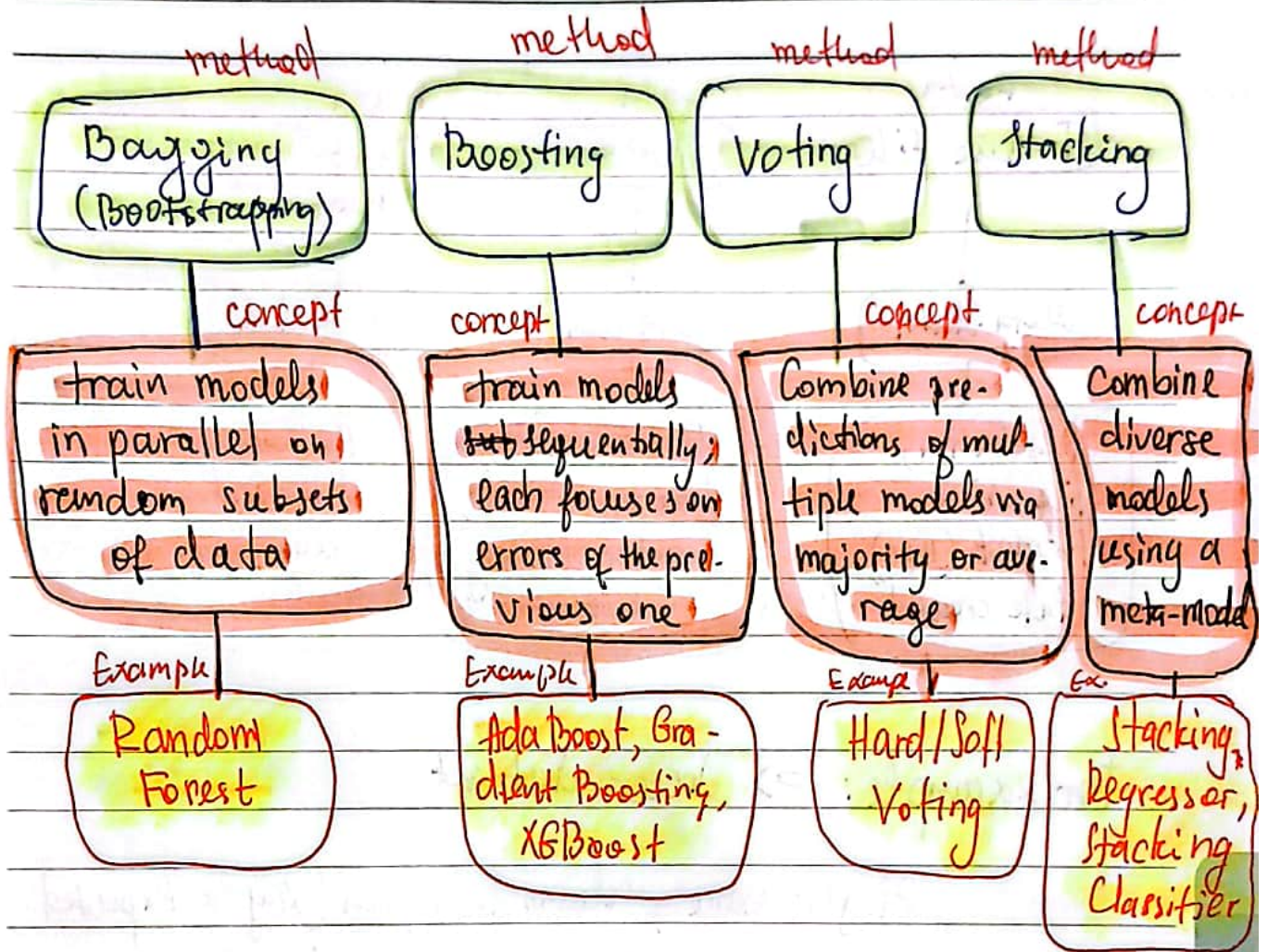
② Ensemble algorithms:

- ②.1 Accuracy
- ②.2 Robustness
- ②.3 Generalization

③ The "No Free lunch Theorem" (David Wolpert and William Macready, 1996)

④ Universal, va Best Algorithm yo'q!

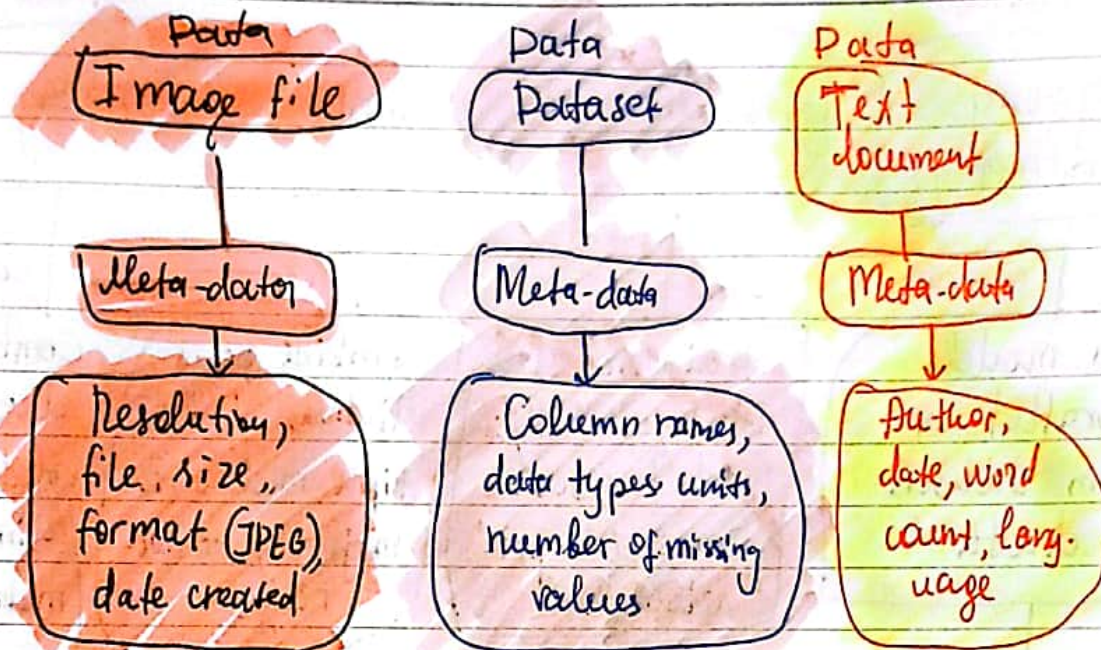
⑤ Har bir algorithm o'zining kuchli va kuchsiz tomonlarga ega va bu data/data taqsimotiga va muammoga bog'liq.



"Meta" yunonchadan kelib biror narsa-
ning "haqida", "artida" degan ma'noni anglatadi.

→ ya'ni biror obyektning o'zi haqida
yuqori darajada fikr yuritish yoki ma'-
lumot berishni anglatadi.

"Metadata" -- data about data (ma'lumot/
haqida ma'lumot)

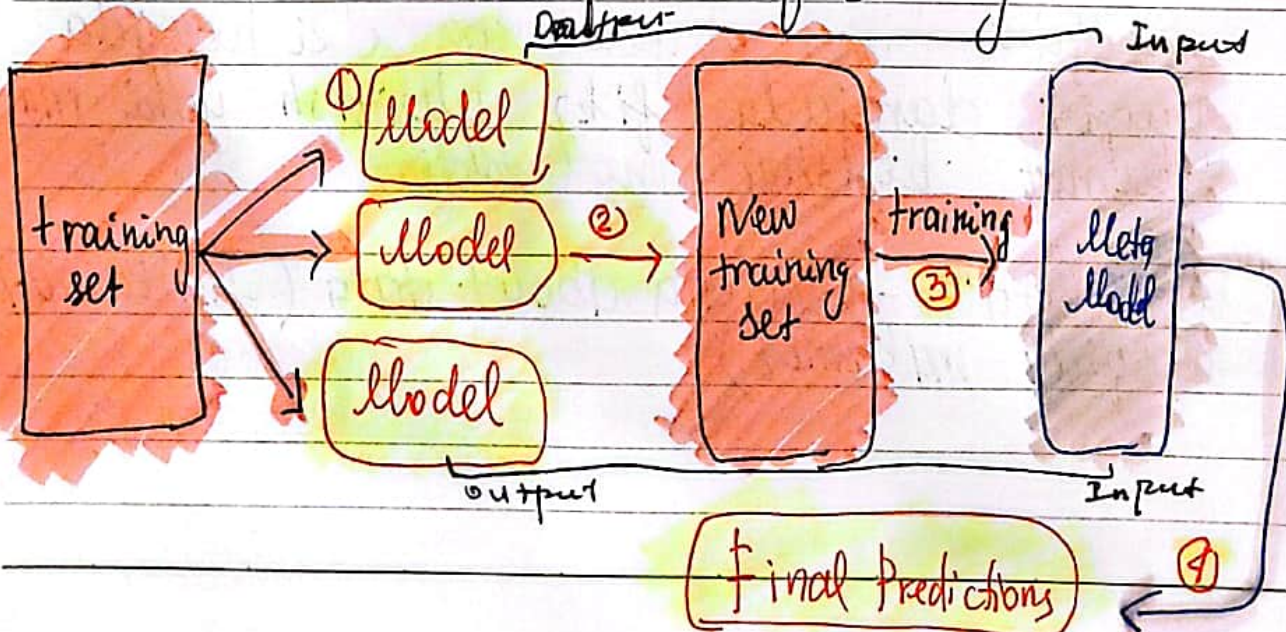


For example: $\Rightarrow$ Soccer dataset

Player name, column description (xGg = Expected goals per game), source of data

• Stacking (Stacked Generalization - 1992 Wolpert)

The process of Stacking



- Base Models (Level 0) $\rightarrow$ asalan bundan natij'a: 0 chiqdi
 $\hookrightarrow$ Train each model (LR, DT, KNN) $\rightarrow$ Bundan 1, 1, 1
 $\hookrightarrow$ Bir xil training data and predictions
- Predictionlarni birlashtirish (Meta-Features) $[0, 1, 1, 1]$

• Prediction/ni birlashtirish (Meta-Features)

% LR = 0.75

% Decision Tree = 0.6

% KNN = 0.9



• Meta-model (Level 1)

$\hookrightarrow$ Base modelarning output/ini Meta-model uchun Input boladi

Input Data



Base Models (Level 0)

Linear Regression

Decision Tree

KNN



(Predictions) $\Rightarrow$ Input/Output/

Meta-model (Level 1)

$\rightarrow$ e.g., Ridge Regression

Final Output

Stacking amaliyot

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
```

Level - 0 (Base Models)

```
base_learners = [
    ("lr", LinearRegression()),
    ("dt", DecisionTreeRegressor(max_depth=5)),
    ("svr", SVR())
]
```

Level - 1 (Meta Model)

```
meta_model = LinearRegression()
```

```
stack = StackingRegressor(estimators=base_learners,
                           final_estimator=meta_model)
```

```
stack.fit(X_train, y_train)
```

```
y_pred = stack.predict(X_test)
```

```
print("Voting Regressor MSE: ", mean_squared_error(
    y_test, y_pred))
```

```
print("Voting Regressor R2 : ", r2_score(y_test, y_pred))
```

Input Data

df

```
xl = df.drop("xG", axis = 1)
```

```
y = df["xG"]
```

```
xl_train, xl_test, y_train, y_test = train_test_split(xl, y, test_size = 0.3, random_state = 42)
```

Base Models (Level 0)

```
base_learners = [
```

```
    ("Linear Regression", LinearRegression()),
```

```
    ("Decision Tree", DecisionTreeRegressor(max_depth=5)),
```

```
    ("SVR", SVR())
```

```
]
```

Predictions

```
base_train_preds = []
```

```
base_test_preds = []
```

```
print(" == Base Models == ")
```


for name, model in base_learners:

```
model.fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```

```
base_train_preds.append(y_train_pred)
base_test_preds.append(y_test_pred)
```

Individually

```
r2_train = model.score(X_train, y_train)
r2_test = model.score(X_test, y_test)
```

```
print(f"[name] - R2 train: {r2_train:.3f},  
test: {r2_test:.3f}")
```

Predictions → Meta features

```
import numpy as np
```

```
X_meta_train = np.column_stack((base_train_preds[0])
```

```
X_meta_test = np.column_stack((base_test_preds[0])
```

```
print(X_meta_train)
```

```
print(X_meta_test)
```

Level -1 (Meta model using predictions)

meta-model = Ridge() # e.g., Ridge Regression as meta-learner

meta-model.fit(X-meta-train, y-train)

y-final-pred = meta-model.predict(X-meta-test)

r2-final = meta-model.score(X-meta-test, y-test)

print("=== Meta Model (Stacking) ===")

print(f"Final Predictions: {y-final-pred[:5]}")

print(f"R² test (final output): {r2-final:.3f}")

Huddi shu tartibda Classifier ham boladi.

• Stacking + :

- ① Accuracyni oshiradi;
- ② Biasni kamaytiradi.
- ③ Complex non-linear xolatda ham yaxshi ishlaydi.

• Stacking - :

- ① Computationally expensive (more training)
- ② Tushunish qiyinroq (Interpretability)
- ③ Overfittingga moyil.

• Qadon foydalanamiz:

- ① Köproq data bilganda
- ② Natijahi yana xam kechaytirgchi bilganingida
- ③ Computational cost ni xal qila olganimizda

• Stacking xulosa:

- ① Modellarni ushlli yo'l usqali birlashtiradi.
- ② Oddiy: voting yoki, orta arifmetik dan yuqori: dardjada ishlaydi...
- ③ Classification va regression uchun ham ishlaydi.
- ④ Base learnerga bog'liq ravishda ishlaydi.

Aspect	Voting	Stacking	Random Forest
Complexity (cost)	Low	Higher	Medium
Learning combination	No	Yes (meta-learner)	No (just aggregation)
Risk of overfitting	Low	Higher	Medium
Performance	Often good	Can be better, but not guaranteed	Usually strong